

Key Dates

Released: September 10, 2026

Due date: **September 24, 2026** (*before 11:59 PM*)

Objectives

This second homework builds on the results of the first and focuses this time on the **interactivity** of the visualization.

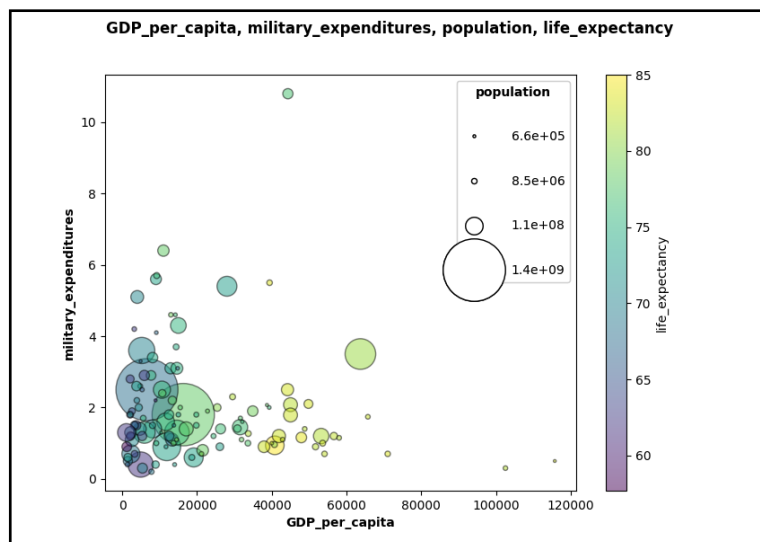
Context

The dataset for this homework is a [World Factbook dataset compiled by the CIA](#). It lists several important attributes about many countries: birth rate, energy consumption, gross domestic product (GDP), health, national debt, and more. As in the first homework, this is a tabular dataset, and we will focus on several interaction mechanisms to facilitate its visualization.

Tasks

Task 1: Bubble Chart

In his famous 2006 [TED Talk](#), Hans Rosling gave an [impressive presentation](#) about global health statistics. For this first task, you will create a basic bubble chart visualization similar to what you did in the second task of the first homework; in other words, a scatter plot in which individual data points are drawn as circles whose *position*, *area*, and *color* vary as a function of four variables.



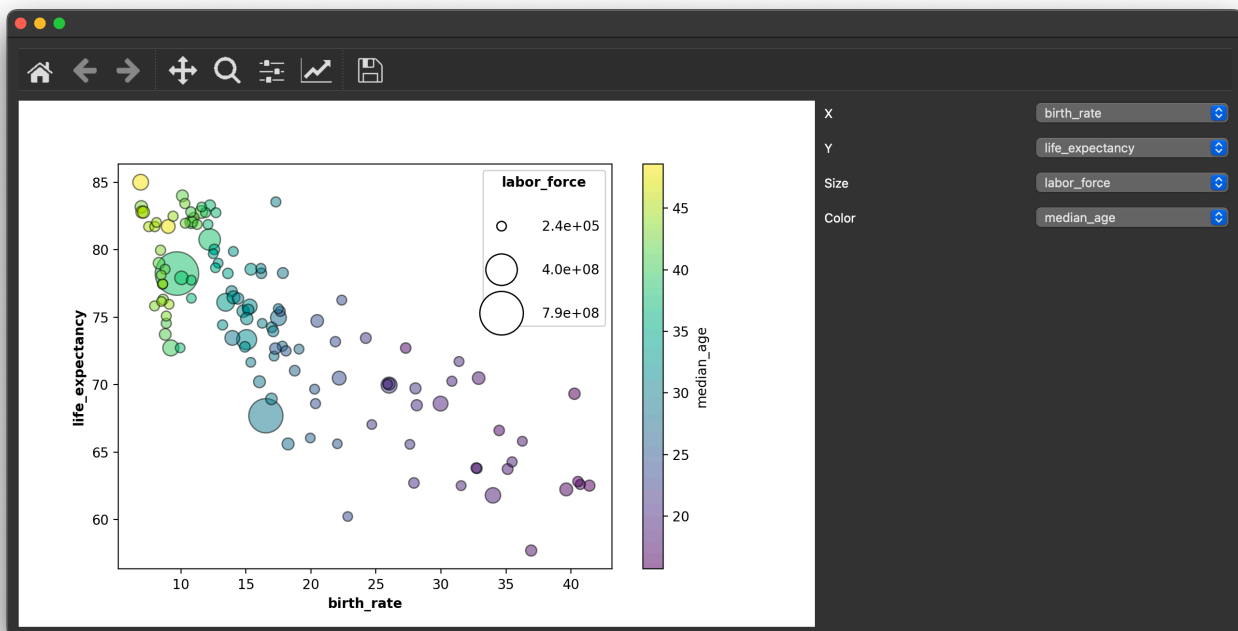
To demonstrate your bubble chart implementation, you will create a program that allows the user to select which attributes to assign to the different degrees of freedom of the bubble chart (x, y, size, and color) on the command line. If no such parameters are provided, your visualization should select "GDP_per_capita" for x, "military_expenditures" for y, "population" for area/size, and "life_expectancy" for color (refer to the image above for an example). Make sure to select scales, ranges, and color maps appropriately. **Include the necessary legends and labels** (see example above). An example showing how to create a legend for the size of the bubbles can be found [here](#).

Deliverable and API: `hw2_bubbles.py -i <dataset path> [-x X] [-y Y] [-s SIZE] [-c COLOR]`

Task 2: Widgets

In the previous task, you were asked to assign specific (and arbitrary) attributes to each parameter (degree of freedom) of your bubble chart. In reality, the visualization would be much more useful if the user could decide at runtime what attributes should be displayed and how. The goal of the second task is to create a user interface that makes this dynamic selection possible. For that, you will augment the visualization of the first task with four [widgets](#) that collectively allow the user to select, for each of x, y, size, and color, what attributes of the dataset should be used. To do so, your widgets should display a list of the existing attributes. Here, each displayed attribute should be selected through a [drop-down menu](#) (or dropdown list). To create these menus, use PyQt, following the example available [here](#).

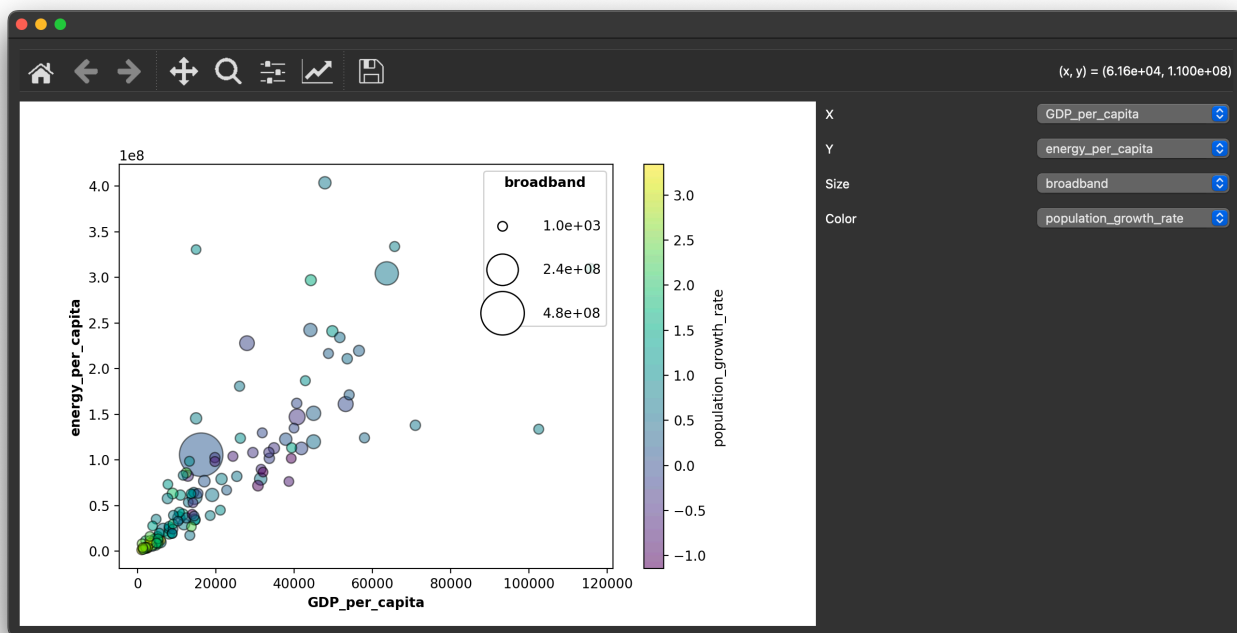
A second aspect of the visualization that the user might want to control interactively is the scaling of the area with respect to the selected attribute. To enable this, you will add a slider bar whose value selection will be tied to the area of the circles in your bubble chart visualization. The previous example also shows how to create a slider bar with PyQt.



Note that your visualization must be automatically updated every time a change is made to any widget selection. Again, **include the necessary legends** and **labels**, and **update them as needed** when changes are made to the drop-down menus or slider bar. As in the previous task, your program should allow the user to initialize the visualization parameters on the command line, defaulting the same attributes as before if none are provided. The user can then interactively modify them.

Deliverable and API: `hw2_widgets.py -i <dataset path> [-x X][-y Y][-s SIZE][-c COLOR]`

The previous and following images show some results obtained for two combinations of attributes. The third image shows the user interface with drop-down menus. Note: the slider bar controlling the size of the bubbles is not shown. It is shown in the images illustrating the next tasks.



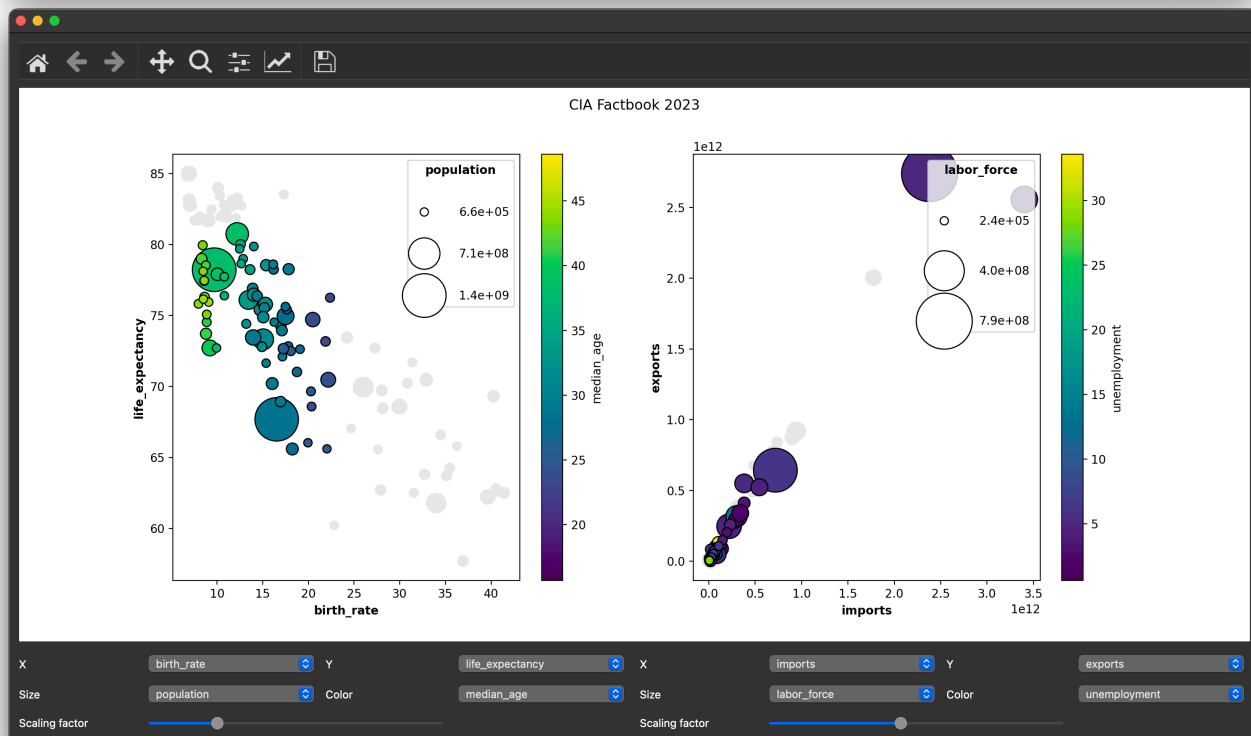
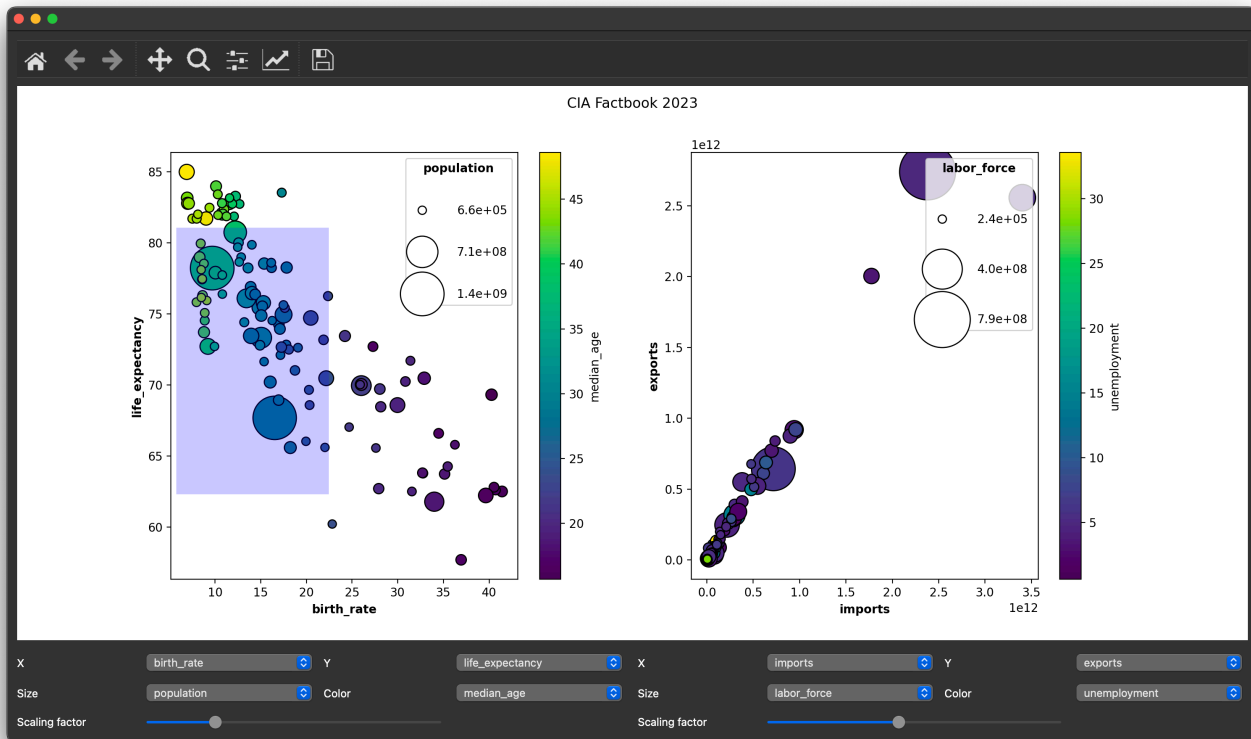
Task 3: Linked Brushing

For this third task, you will display two bubble charts side by side, which will allow you to visualize up to twice as many attributes than what is possible with a single bubble chart. The user will be free to select different sets of attributes to control the appearance of each chart. Note that you will need to double the number of widgets created in the previous task.

One issue with this visualization is understanding how data points and patterns present in one chart relate to what can be seen in the other chart. A possible solution would be to select the same attributes to control the bubbles' radius and/or color in all charts. A more flexible approach involves what is known as *linked brushing*. Using linked brushing, the local selection of some data points in a visualization triggers the selection of the same data points in another visualization. Your task is to implement this method. Make sure a selection in one window is replicated in the other window. Practically, your implementation should allow for data points to be selected within a rectangular region

drawn by the user. This selection should be visually accompanied by a change in the appearance of the selected items in both windows to make them easy to spot. Brushing can be implemented in Matplotlib using its [RectangleSelector](#). As in the previous task, you are free to select the attributes of your choice to initialize both bubble charts. Observe that the **selected points are always drawn in front** of the excluded data points, and not hidden by them.

Deliverable and API: `hw2_brushing.py -i <dataset path>`



The images above show an example of linked brushing, first the region selection in the left visualization using a rectangle selector, then the resulting differential display of data points in both bubble charts. Note the change in the appearance of the data points not included in the selection.

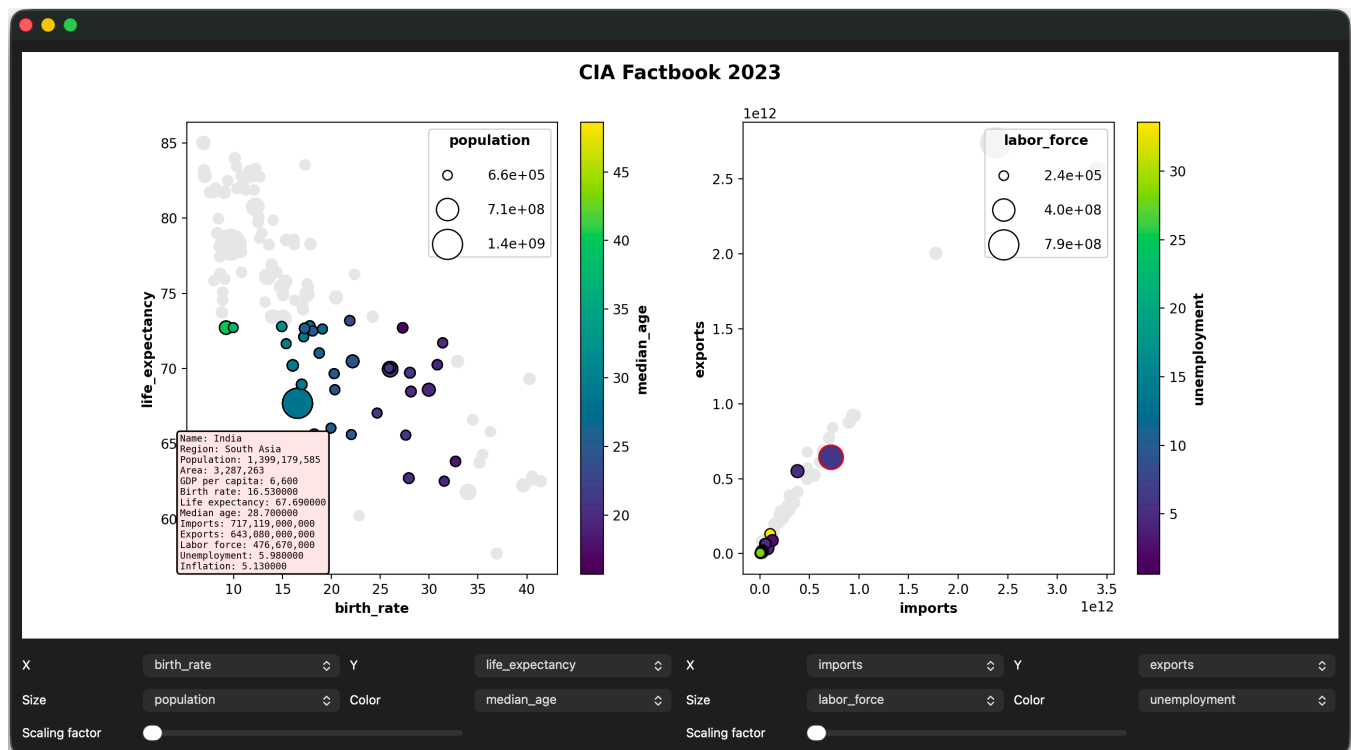
Task 4: Details on Demand

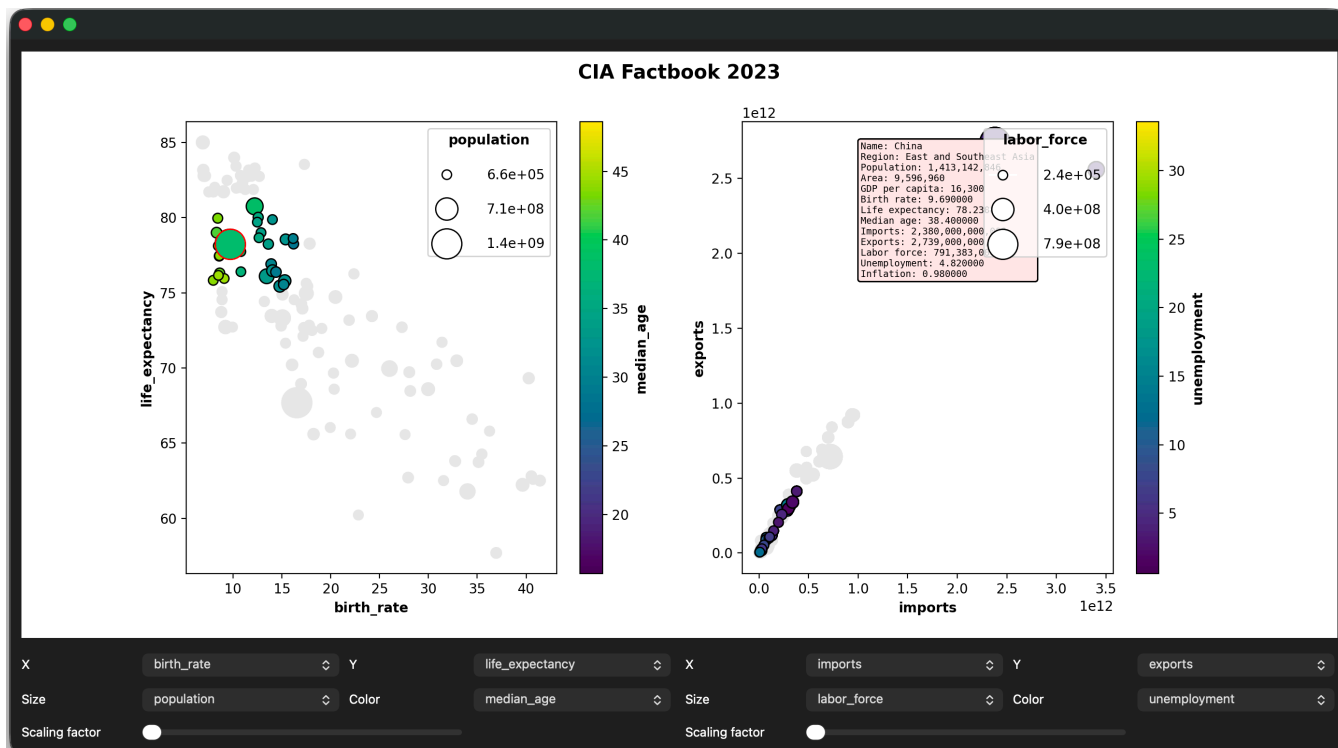
Though two bubble charts can show a substantial number of attributes at once, they can only convey a relatively small portion of the information available in this dataset. A typical means to overcome this limitation is to allow the user to query additional details about the data points shown in the visualization (so-called "details-on-demand" in the language of [Shneiderman's mantra](#)).

To access additional details, you will use the *hover* event mechanism to trigger the display of many of the attributes associated with a given data point in a *tooltip*. Specifically, when the mouse hovers over a particular data point, a window should appear to the side of the corresponding location, displaying the name of the country along with a series of (*attribute name, value*) pairs that **show a subset** of the information available for this country. In addition, the country selected should be **highlighted** in both charts. Your fourth task is to add this feature to the visualization tool you created in Task 3. In particular, your visualization should also support brushing and linking. As a starting point for this task, you might find it helpful to refer to the explanations given [here](#) about the use of event handling to detect hovering selections.

Deliverable and API: `hw2_tooltip.py -i <dataset path>`

The following images show tooltip examples. Note the red circle highlighting the selected point in the other visualization.





Data Set

The dataset is available in CSV format [CIA world factbook 2023.csv](#).

Note: This is only part of the (now retired) [CIA's World Factbook](#).

Submission

Submit your solution for this homework on Gradescope before **September 24, 2026, at 11:59 pm**.

The instructions below echo the ones provided for the first homework.

- Include all the requested Python code files: hw2_bubbles.py, hw2_widgets.py, hw2_brushing.py, hw2_tooltip.py
- Include all submitted files in a single directory named homework2. zip and submit that directory as your solution. **Do not include the data file** in your submission.
- Test your programs on the **command line** and make sure that they satisfy the prescribed API.
- Fill out the AI disclosure form for this homework.

Last modified September 10, 2026